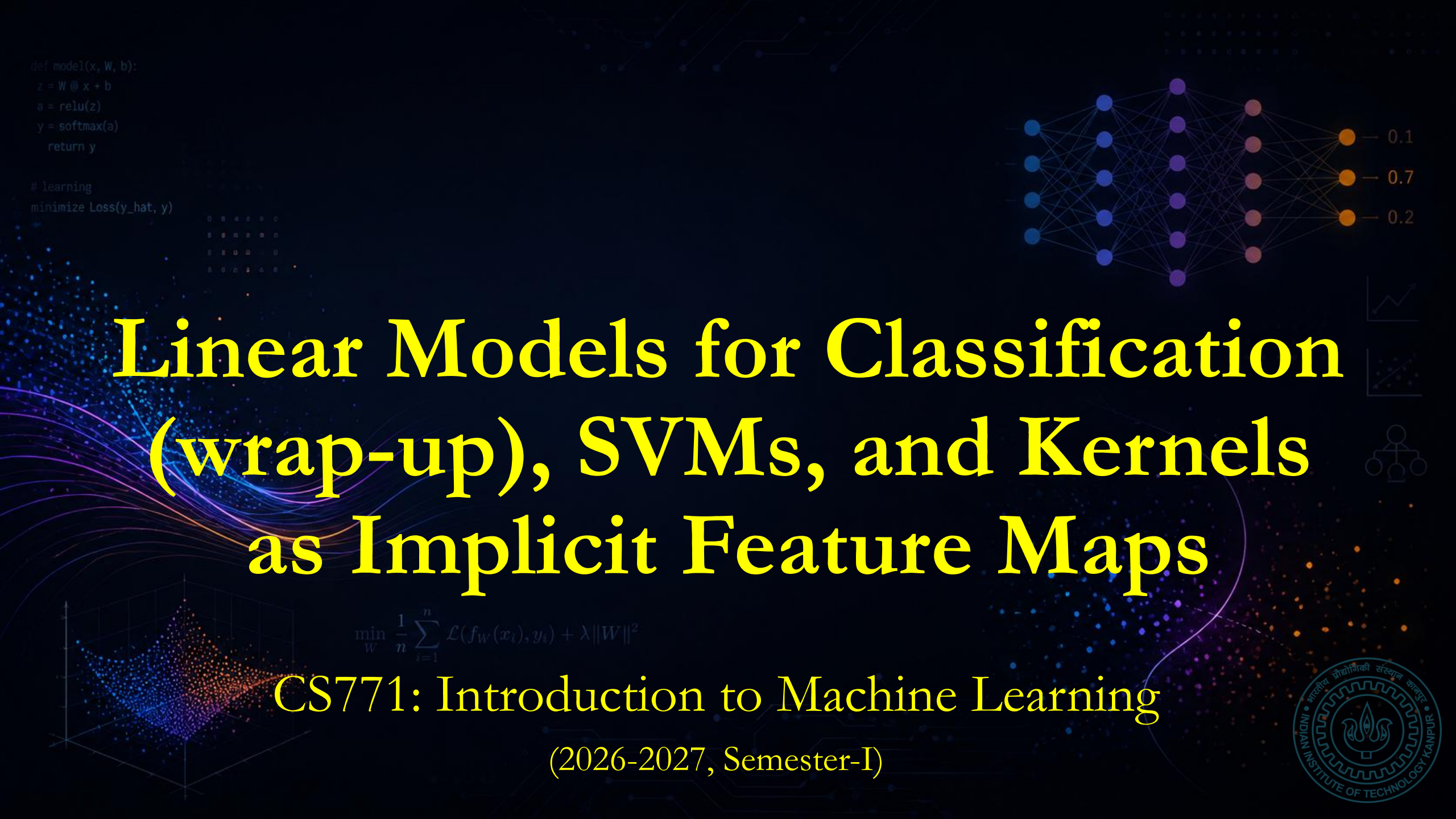


Linear Models for Classification (wrap-up), SVMs, and Kernels as Implicit Feature Maps

CS771: Introduction to Machine Learning

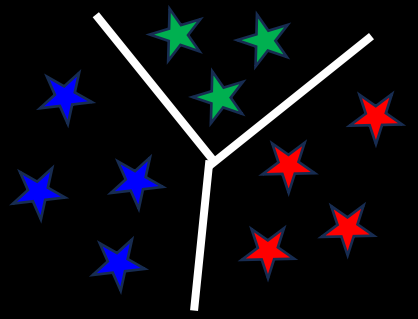
(2026-2027, Semester-I)

From One Score to C Scores

The binary classification case had one score for $\boldsymbol{x}_n$: $s_n = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n + b$

In multi-class classification with $C > 2$ classes $\boldsymbol{x}_n$ has C scores as

$$s_{n,i} = \boldsymbol{w}_i^\mathsf{T}\boldsymbol{x}_n + b_i \qquad i = 1,2,\ldots,C$$



Collectively $\boldsymbol{s}_n = \boldsymbol{W}^\mathsf{T}\boldsymbol{x}_n + \boldsymbol{b} \in \mathbb{R}^C$

Prediction: $\hat{y}_n = \operatorname{argmax}_i s_{n,i}$

$$\boldsymbol{s}_n = \begin{bmatrix} -0.8 \\ 2.1 \\ 0.7 \end{bmatrix} \Rightarrow \hat{y}_n = 2$$

In "one-hot" notation

$$\hat{y}_n = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Multi-class decision boundaries: The boundary between classes i and j occurs where their scores are equal:

$$s_i(\boldsymbol{x}) = s_j(\boldsymbol{x})$$

$$(\boldsymbol{w}_i - \boldsymbol{w}_j)^\mathsf{T}\boldsymbol{x} + (b_i - b_j) = 0$$

Thus, unlike binary classification, multiple pairwise decision boundaries can exist. Likewise, multiple class-wise margins can be defined by comparing each true-class score with competing class' scores.

C Scores to C Class Probabilities

We can map the scores $\boldsymbol{s}_n = [s_{n,1}, s_{n,2}, \dots, s_{n,C}]$ to a vector $\hat{\boldsymbol{p}}_n$ of estimated probabilities of $\boldsymbol{x}_n$ belonging to each of the C classes

$$\hat{\boldsymbol{p}}_n = g(\boldsymbol{s}_n) = [\hat{p}_{n,1}, \hat{p}_{n,2}, \dots, \hat{p}_{n,C}] \quad \text{s.t.} \quad \hat{p}_{n,i} \geq 0 \text{ and } \sum_{i=1}^{C} \hat{p}_{n,i} = 1$$

Multi-class logistic (popularly known as softmax regression) using the softmax function to convert score vector $\boldsymbol{s}_n$ to class probabilities

$$\hat{p}_{n,i} = \frac{\exp(s_{n,i})}{\sum_{j=1}^{C} \exp(s_{n,j})}$$

$$s_{n,i} > s_{n,k} \Rightarrow \hat{p}_{n,i} \geq \hat{p}_{n,k}$$

$$\boldsymbol{s}_n = \begin{bmatrix} 1 \\ 2 \\ .0.1 \end{bmatrix} \Rightarrow \hat{\boldsymbol{p}}_n \approx \begin{bmatrix} 0.24 \\ 0.66 \\ 0.10 \end{bmatrix}$$

$$\text{argmax}_i \, s_{n,i} = \text{argmax}_i \, \hat{p}_{n,i}$$

Learning the Multi-class Classification Model

Most binary classification losses have multi-class extensions

For an input $\boldsymbol{x}_n$ with true $y_n \in \{1,2,\ldots,C\}$, score $\boldsymbol{s}_n = \boldsymbol{W}^\top \boldsymbol{x}_n + \boldsymbol{b}$, and class probabilities $\hat{p}_{n,i} = \frac{\exp(s_{n,i})}{\sum_{j=1}^{C} \exp(s_{n,j})}$, the analogue of logistic loss is

$$\mathcal{L}(\boldsymbol{W}, b) = -\frac{1}{N}\sum_{n=1}^{N} \log \hat{p}_{n,y_n} + \lambda \|\boldsymbol{W}\|_F^2$$

Loss function for the softmax regression model with ℓ_2 regularization on the weight matrix $\boldsymbol{W} = [\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_C]^\top$

We can optimize it using GD/SGD or variants just like we optimize the loss functions for binary classification

For the gradient w.r.t. $\boldsymbol{w}_i$, the example $(\boldsymbol{x}_n, y_n)$ contributes proportional to the error in the prediction for class i

Ignoring the regularizer term, the gradient $\nabla_{\boldsymbol{w}_i} \mathcal{L} = \frac{1}{N}\sum_{n=1}^{N} (\hat{p}_{n,i} - \mathbb{I}[y_n = i])\, \boldsymbol{x}_n$

(Multi-class) Classification in Modern AI

LLMs have next-token prediction as a key task

- Each possible token in the vocabulary can be viewed as a class
- Given a context representation $\boldsymbol{h}$, the model computes scores

$$s_i = \boldsymbol{w}_i^\top \boldsymbol{h} + b_i \qquad i = 1,2,\ldots,V$$

V is the vocabulary size

- Softmax converts the V scores to next-token probabilities:

$$\hat{p}_i = \frac{\exp(s_i)}{\sum_{j=1}^{V} \exp(s_j)}$$

In some applications, the number of classes C is huge

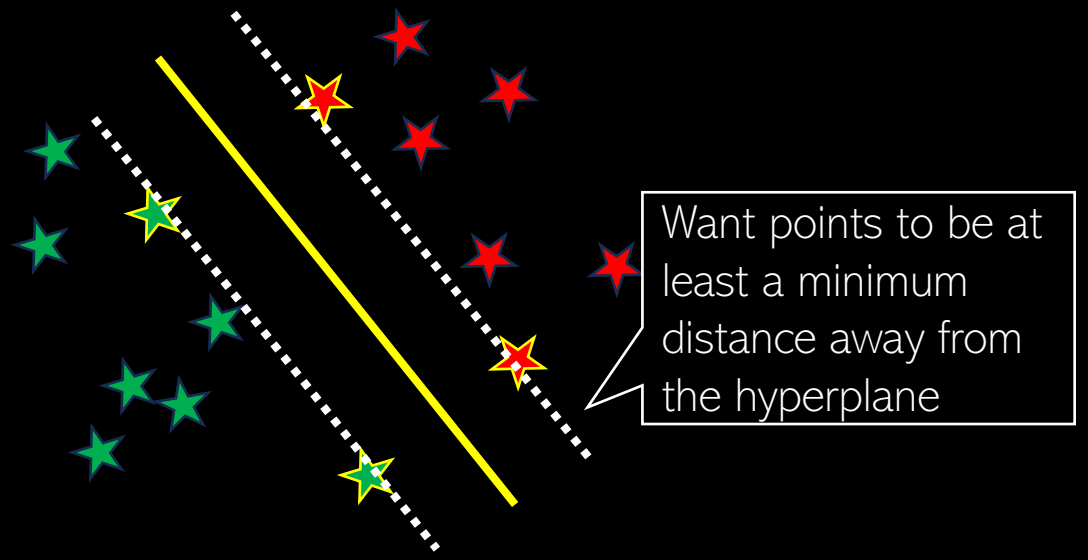
- Standard training and prediction will be $O(C)$
- Methods exist to speed up (e.g., hierarchical softmax)

Later

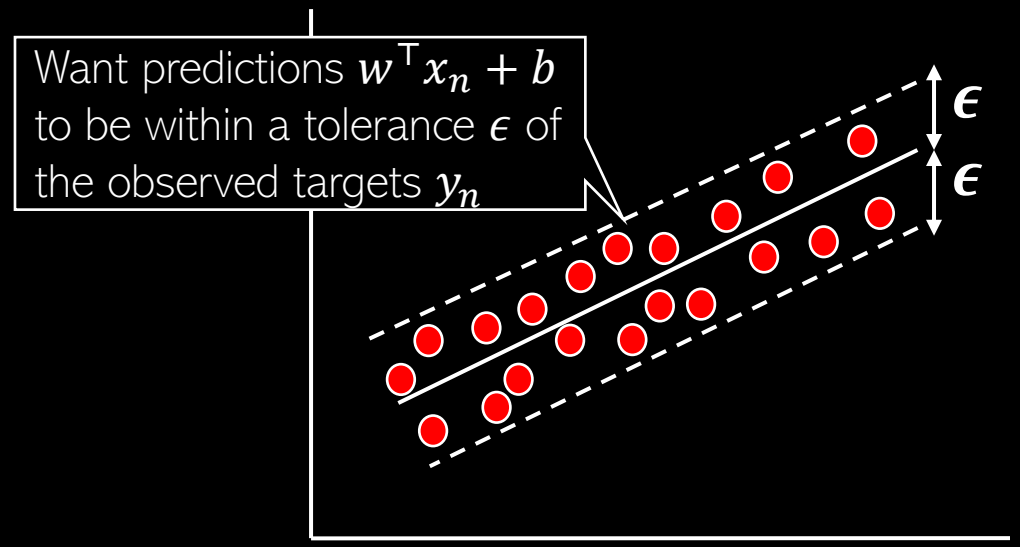


Learning with Margins

Classification with margins



Regression with margins



Learning with margins can be posed as a constrained optimization problem (e.g., constraints on distance from hyperplane, or constraint on prediction error on each training input)

A Maximum Margin Classifier: SVM

Margin of the hyperplane is geometric margin of the closest example

$$\gamma = \min_n \frac{y_n(\boldsymbol{w}^\top \boldsymbol{x_n} + b)}{\|\boldsymbol{w}\|} = \frac{1}{\|\boldsymbol{w}\|}$$

Since scaling of $(\boldsymbol{w}, b)$ doesn't change the hyperplane, we can assume them to be such that $y_n(\boldsymbol{w}^\top \boldsymbol{x_n} + b) = 1$ for the closest example $(\boldsymbol{x}_n, y_n)$

Thus hyperplane with maximum margin is one with smallest $\|\boldsymbol{w}\|$

Thus solving this objective will give us such a max-margin hyperplane

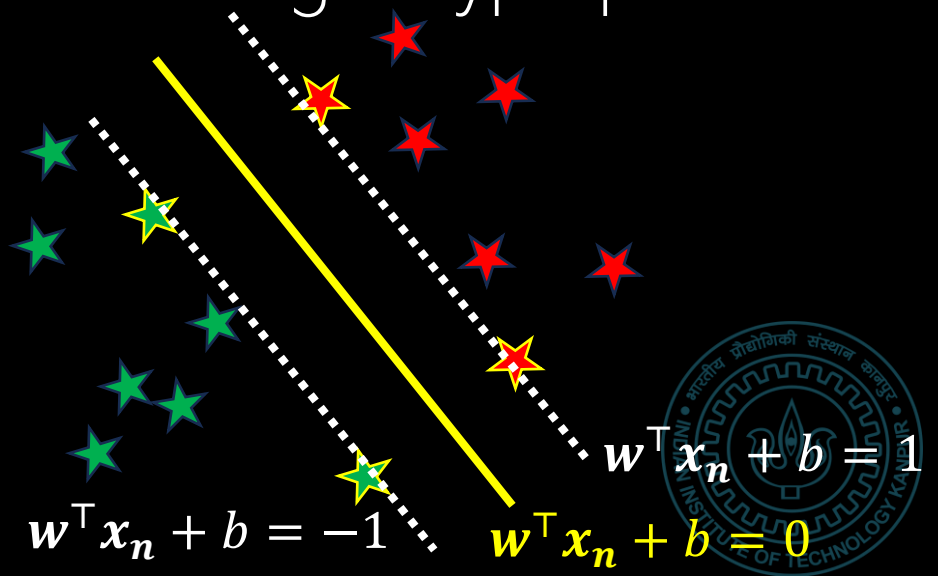
$$\min_{\boldsymbol{w},b} \frac{\|\boldsymbol{w}\|^2}{2}$$

$$\text{s.t. } y_n(\boldsymbol{w}^\top \boldsymbol{x_n} + b) \geq 1 \quad \forall n$$

Can construct the Lagrangian (exercise) and solve in dual

Called "hard-margin" SVM – no training inputs allowed to fall inside the margin region or be misclassified

Can show that $\boldsymbol{w}^{(\text{opt})} = \sum_{i=1}^{N} \alpha_i \, y_i \boldsymbol{x}_i$ (consistent with representer theorem) where α_i's are Lagrange multipliers



KKT Conditions and Support Vectors

Consider a constrained optimization problem

$$\boldsymbol{w}^{opt} = \operatorname{argmin}_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) \quad \text{s.t.} \quad g_n(\boldsymbol{w}) \leq 0 \quad n = 1,2,\ldots,N$$

The Lagrangian $\mathcal{J}(w,\alpha) = \mathcal{L}(\boldsymbol{w}) + \sum_{n=1}^{N} \alpha_n \boldsymbol{g_n}(\boldsymbol{w})$ where $\alpha_n \geq 0$

At the optimum $(\boldsymbol{w}, \boldsymbol{\alpha})$, we must have $\alpha_n \boldsymbol{g_n}(\boldsymbol{w}) = 0$

Called "complementary slackness" (one of the KKT conditions

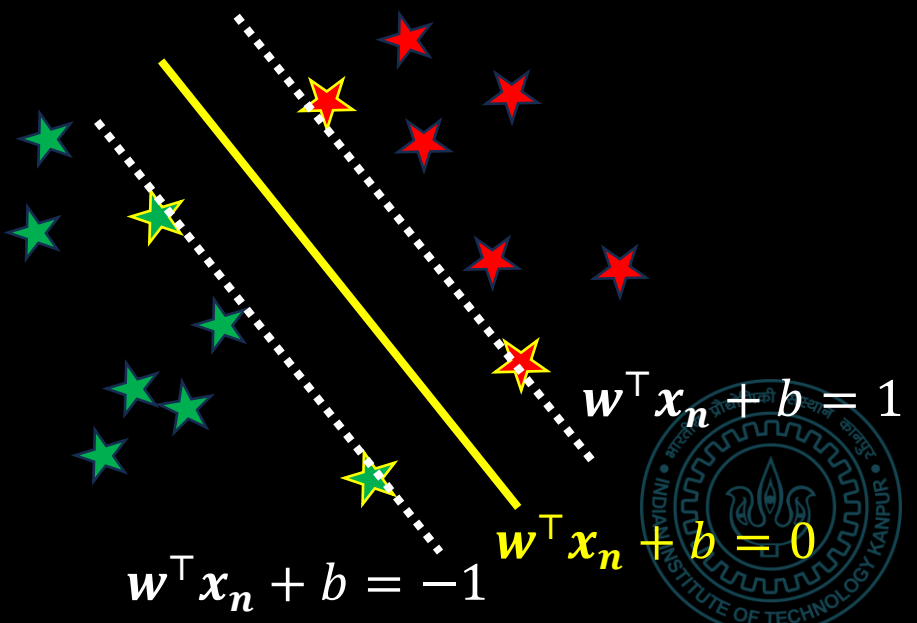
In the context of SVMs, this means

$$\alpha_n[1 - y_n(\boldsymbol{w}^\top \boldsymbol{x_n} + b)] = 0$$

$$\Rightarrow \alpha_n > 0 \text{ only if } y_n(\boldsymbol{w}^\top \boldsymbol{x_n} + b) = 1$$

Thus, for SVM, very few α_n's are non-zero; in particular for points with $y_n(\boldsymbol{w}^\top \boldsymbol{x_n} + b) = 1$

These are the points on the margin boundaries on both sides and are called support vectors



SVM with Margin Violations

Often desirable to have marginal violation by some training inputs.
"Violation" means falling inside margin region or crossing to wrong side

This violation based formulation is called the "soft-margin" SVM

Goal: Maximize the margin AND minimize the sum of violations

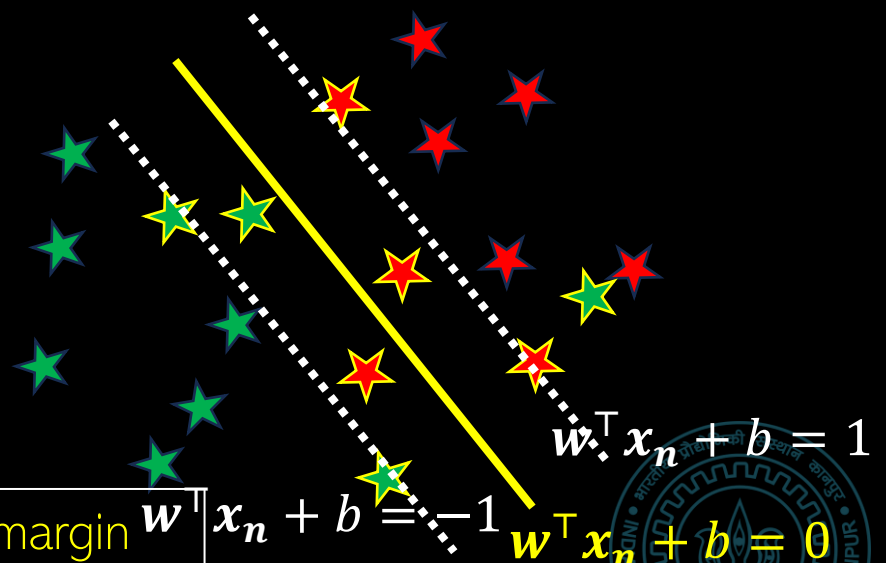
Trade-off hyperparam

$$\min_{w,b,\xi} \frac{\|w\|^2}{2} + C\sum_{n=1}^{N} \xi_n$$

s.t. $y_n(w^\top x_n + b) \geq 1 - \xi_n \quad \forall n$

$\xi_n \geq 0 \quad \forall n$

Can construct the Lagrangian and solve in dual (the Lagrangian shown on the next slide)



Support vectors: Points that fall exactly on the margin boundary or violate the margin boundary (lie inside the margin region or cross over to the side of the other class)

Soft-Margin SVM

The Lagrangian for the soft margin SVM problem can be written as

$$\min_{w,b,\xi} \max_{\alpha \geq 0, \beta \geq 0} \frac{\|w\|^2}{2} + C\sum_{n=1}^{N} \xi_n + \sum_{n=1}^{N} \alpha_n \{1 - y_n(w^\top x_n + b) - \xi_n\} - \sum_{n=1}^{N} \beta_n \xi_n$$

Setting gradient of the above w.r.t. w as zero gives $w = \sum_{i=1}^{N} \alpha_i y_i x_i$

Substituting $w = \sum_{i=1}^{N} \alpha_i y_i x_i$ and after further simplification gives the objective in terms of the dual vars α_n, β_n (exercise)

Even β_n's disappear!

Just need to solve for the α's

$$\max_{0 \leq \alpha \leq C} \sum_{n=1}^{N} \alpha_n - \frac{1}{2}\sum_{m=1}^{N}\sum_{n=1}^{N} \alpha_m \alpha_n y_m y_n x_m^\top x_n$$

Many ways to solve it; co-ordinate wise maximization with a projection step to maintain the constraint $0 \leq \alpha_n \leq C$ (try this as an exercise) is a very popular and efficient method

Once we have α_n's, we can recover $w = \sum_{i=1}^{N} \alpha_i y_i x_i$

Soft-Margin SVM = Hinge Loss + L2 Regularization

Slack terms ξ_n's are present as $y_n(\boldsymbol{w}^\top \boldsymbol{x_n} + b) \geq 1 - \xi_n$ and $\xi_n \geq 0$

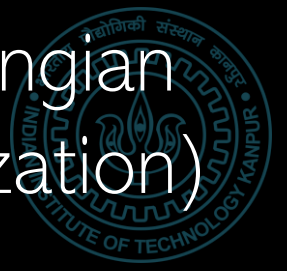
Thus each ξ_n is simply the hinge loss: $\xi_n = \max\{0, 1 - y_n(\boldsymbol{w}^\top \boldsymbol{x}_n + b)\}$

Since soft-margin SVM minimizes the sum of slacks and maximizes the margin (same as minimizing the ℓ_2 norm of $\boldsymbol{w}$), the overall objective

$$\mathcal{L}(\boldsymbol{w}, b) = \sum_{n=1}^{N} \max\{0, 1 - y_n(\boldsymbol{w}^\top \boldsymbol{x}_n + b)\} + \frac{\lambda}{2} \boldsymbol{w}^\top \boldsymbol{w}$$

Thus we can also minimize the above loss to learn an SVM classifier

This is sometimes called solving the SVM in the primal (no Lagrangian construction needed for this; just a regularized hinge loss minimization)



Multiclass SVMs, Nonlinear SVM, SVM Regression

SVM (large-margin) extensions exists for multi-class classification too

Maintain C weights $\boldsymbol{W} = [\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_C]$. The max-margin problem

$$\boldsymbol{W}^{(\text{opt})} = \text{argmin}_{\boldsymbol{W}} \sum_{c=1}^{C} \frac{\|\boldsymbol{w}_c\|^2}{2} \quad \text{s.t.} \quad \boldsymbol{w}_{y_n}^\top \boldsymbol{x}_n \geq \boldsymbol{w}_c^\top \boldsymbol{x}_n + 1 \quad \forall k \neq y_n \quad \forall n$$

Score on correct class

Score on incorrect class

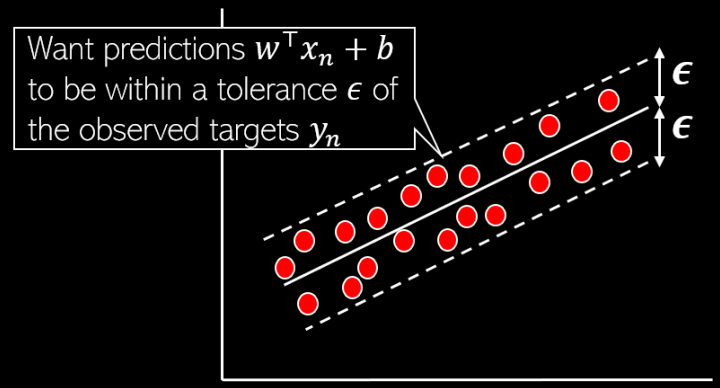
Nonlinear SVMs can be designed by replacing $\boldsymbol{x}$ with $\boldsymbol{\phi}(\boldsymbol{x})$

Explicitly constructed $\boldsymbol{\phi}$ or implicitly constructed $\boldsymbol{\phi}$ using kernels (discussed next)

SVMs for regression can be designed using an objective such as

Minimize $\frac{\|\boldsymbol{w}\|^2}{2}$

Subject to $|y_n - \boldsymbol{w}^\top \boldsymbol{x}_n - b| \leq \epsilon$



Kernel Functions as Implicit Feature Maps



Feature Map + Dot Product = Kernel Function

Consider two inputs $\boldsymbol{x} = (x_1, x_2)^\top$ and $\boldsymbol{z} = (z_1, z_2)^\top$ in a 2-dim input space and a feature map $\boldsymbol{\phi}$ such that

$$\boldsymbol{\phi}(\boldsymbol{x}) = \left(x_1^2, \sqrt{2}x_1x_2, x_2^2\right)^\top$$

The dot product $\boldsymbol{\phi}(\boldsymbol{x})^\top\boldsymbol{\phi}(\boldsymbol{z}) = \left(x_1^2, \sqrt{2}x_1x_2, x_2^2\right)^\top \left(z_1^2, \sqrt{2}z_1z_2, z_2^2\right)$

$$= x_1^2z_1^2 + x_2^2z_2^2 + 2x_1x_2z_1z_2$$

$$= (x_1z_1 + x_2z_2)^2$$

$$= (\boldsymbol{x}^\top\boldsymbol{z})^2$$

$$= k(\boldsymbol{x}, \boldsymbol{z})$$

Using $k(\boldsymbol{x}, \boldsymbol{z})$ there is no need to explicitly define and compute $\boldsymbol{\phi}(\boldsymbol{x}) \in \mathbb{R}^P$ and no need to explicitly compute the dot product $\boldsymbol{\phi}(\boldsymbol{x})^\top\boldsymbol{\phi}(\boldsymbol{z})$ in the $\boldsymbol{\phi}$ space

If $\boldsymbol{x} \in \mathbb{R}^D$, then the cost of computing $k(\boldsymbol{x}, \boldsymbol{z})$ is $O(D)$ whereas computing $\boldsymbol{\phi}(\boldsymbol{x})$ and $\boldsymbol{\phi}(\boldsymbol{x})^\top\boldsymbol{\phi}(\boldsymbol{z})$ has $O(P)$ cost

Savings are huge if $P \gg D$

Very useful idea. Different choices of $k(\boldsymbol{x}, \boldsymbol{z})$ can give us different feature maps $\boldsymbol{\phi}(\boldsymbol{x})$

Choosing $k(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^\top\boldsymbol{z})^2$ is equivalent to choosing $\boldsymbol{\phi}(\boldsymbol{x}) = \left(x_1^2, \sqrt{2}x_1x_2, x_2^2\right)^\top$

Another Kernel

Radial Basis Function (RBF)/Gaussian kernel with bandwidth hyperparameter γ

Thus kernels enable us to use very high-dim ϕ without even computing ϕ or its dot products



Which $\phi(x)$ arises due to $k(x, z) = \exp[-\gamma \|x - z\|^2]$?

Assuming $\gamma = 1$ and x, z to be scalars

$$k(x,z) = \exp[-(x-z)^2]$$

$$= \exp(-x^2)\exp(-z^2)\exp(2xz)$$

$$= \exp(-x^2)\exp(-z^2)\sum_{k=0}^{\infty}\frac{2^k x^k z^k}{k!}$$

$$= \sum_{k=0}^{\infty}\frac{2^k x^k \exp(-x^2) z^k \exp(-z^2)}{k!}$$

$$= \phi(x)^{\top}\phi(z)$$

Thus $\phi(x)$ is an infinite-dim vector (ignoring the constants coming from the 2^k and $k!$ Terms)

$$\phi(x) = \begin{bmatrix} \exp(-x^2)x^0 \\ \exp(-x^2)x^1 \\ \exp(-x^2)x^2 \\ \vdots \\ \vdots \\ \exp(-x^2)x^{\infty} \end{bmatrix}$$

Thus using RBF kernel is equivalent to using an inf-dim $\phi(x)$

But constructing $\phi(x)$ or compute $\phi(x)^{\top}\phi(z)$ not needed – evaluating the function $k(x, z) = \exp[-(x - z)^2]$ does both for us! ☺

Other Popular Kernels

List here is not exhaustive. Many others, such as cosine similarity (kernel), histogram kernel, Laplacian kernel, Matern kernel, Fisher kernel, and many others



Linear kernel: $k(\boldsymbol{x}, \boldsymbol{z}) = \boldsymbol{x}^\top \boldsymbol{z}$

Polynomial kernel of degree d: $k(\boldsymbol{x}, \boldsymbol{z}) = (1 + \boldsymbol{x}^\top \boldsymbol{z})^d$

With $p = 24$, can use it to compare hours (0-23) of a day

Tanh kernel: $k(\boldsymbol{x}, \boldsymbol{z}) = \tanh(\alpha \boldsymbol{x}^\top \boldsymbol{z} + c)$

Periodic kernel: $k(x, z) = \exp\left\{-\frac{2\sin^2(\pi(x-z)/p)}{\ell^2}\right\}$

p is the period, ℓ denotes how rapidly similarity decreases within each period

Each of these implicitly defines a feature mapping

Most kernels have hyperparameters too (e.g., RBF has bandwidth) which have to be set appropriately (e.g., cross-validation)

Kernels can be composed too. E.g., sum of kernels, product of kernels, etc can be used as kernel (e.g., $k_1 + k_2, k_1 \times k_2, k_1 + k_2 \times k_3$, etc)

Nonlinear Models via Kernelization

Recall that when training the SVM in the dual, we have to solve

$$\max_{0 \le \alpha \le C} \sum_{n=1}^{N} \alpha_n - \frac{1}{2} \sum_{m=1}^{N} \sum_{n=1}^{N} \alpha_m \alpha_n y_m y_n \boldsymbol{x}_m^\top \boldsymbol{x}_n$$

Can replace $\boldsymbol{x}_m^\top \boldsymbol{x}_n$ by $\phi(x_m)^\top \phi(x_n)$ using a nonlinear ϕ of our choice

This dot product is basically a kernel/similarity function s.t.

$$k(x_m, x_n) = \phi(x_m)^\top \phi(x_n)$$

Take a linear model like linear SVM and replace standard dot products by kernel

Turn a linear model into a nonlin model like this is called "kernelization"

ML problems where the inputs appear as dot products in the training objective, or learning algo, or prediction function can use this trick

Prediction using Kernelized Models

Or $\boldsymbol{w} = \sum_{n=1}^{N} \alpha_n y_n \boldsymbol{\phi}(\boldsymbol{x}_n)$

For a kernelized (nonlinear) model: $\boldsymbol{w} = \sum_{n=1}^{N} \alpha_n \boldsymbol{\phi}(\boldsymbol{x}_n)$

Assume the map $\boldsymbol{\phi}(\boldsymbol{x}_n)$ is P-dimensional

Two ways to compute score of a test input $\boldsymbol{x}_*$

For some models such as SVMs, only some α_n's are nonzero, so in practice the cost is less than $O(N)$

In general, the cost is $O(N)$

Approach 1

$$s_* = \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_*) = \sum_{i=1}^{P} w_i \phi_i(\boldsymbol{x}_*)$$

If P is finite and $P < N$ then this approach is preferred over approach 2

Approach 2

$$s_* = \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_*) = \sum_{n=1}^{N} \alpha_n \boldsymbol{\phi}(\boldsymbol{x}_n)^\top \boldsymbol{\phi}(\boldsymbol{x}_*) = \sum_{n=1}^{N} \alpha_n k(\boldsymbol{x}_n, \boldsymbol{x}_*)$$

Kernels approximations using low-dim features

Recall that the kernel function assumes that

$$k(\boldsymbol{x}_n, \boldsymbol{x}_m) = \phi(\boldsymbol{x}_n)^\top \phi(\boldsymbol{x}_m)$$

For a kernel k, we can construct (and for some kernels, there naturally exists) an L-dim feature vector $\psi(\boldsymbol{x})$ such that

$$k(\boldsymbol{x}_n, \boldsymbol{x}_m) \approx \psi(\boldsymbol{x}_n)^\top \psi(\boldsymbol{x}_m)$$

While still capturing nonlinearity given by this kernel

Thus, we can use $\psi(\boldsymbol{x}_n) \in \mathbb{R}^L$ within a linear model with $\boldsymbol{w} \in \mathbb{R}^L$

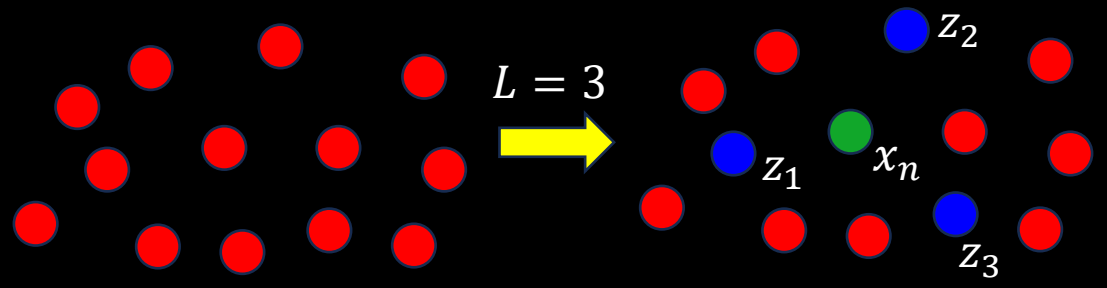
If $L \ll P$ and $L \ll N$, using $\psi(\boldsymbol{x}_n)$'s is more efficient (both training and prediction) than using $\phi(\boldsymbol{x}_n)$'s or $k(\boldsymbol{x}_n, \boldsymbol{x}_m) = \phi(\boldsymbol{x}_n)^\top \phi(\boldsymbol{x}_m)$

For many kernel functions k, there are ways to extract "good" $\psi(\boldsymbol{x}_n)$



Kernels approximations using low-dim features

Approach 1: Pick L "landmark" inputs $\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_L$ in training data



Kernel based similarities of $\boldsymbol{x}_n$ with the L landmarks inputs used as features to construct $\psi(\boldsymbol{x}_n)$

$$\psi(\boldsymbol{x}_n) = [k(\boldsymbol{z}_1, \boldsymbol{x}_n), k(\boldsymbol{z}_2, \boldsymbol{x}_n), k(\boldsymbol{z}_3, \boldsymbol{x}_n)] \in \mathbb{R}^3$$

This approach is called "kernel random features" (since $\boldsymbol{w}$'s are random)

Approach 2: Use the idea* that many kernels can be written as

$$k(\boldsymbol{x}_n, \boldsymbol{x}_m) = \phi(\boldsymbol{x}_n)^\top \phi(\boldsymbol{x}_m)$$

$$= \mathbb{E}_{\boldsymbol{w} \sim p(\boldsymbol{w})}[t_{\boldsymbol{w}}(\boldsymbol{x}_n) t_{\boldsymbol{w}}(\boldsymbol{x}_m)]$$

$$\approx \frac{1}{L} \sum_{i=1}^{L} t_{\boldsymbol{w}_i}(\boldsymbol{x}_n) t_{\boldsymbol{w}_i}(\boldsymbol{x}_m)$$

$$= \psi(\boldsymbol{x}_n)^\top \psi(\boldsymbol{x}_m) \rightarrow \psi(\boldsymbol{x}_n) = \frac{1}{\sqrt{L}} [t_{\boldsymbol{w}_1}(\boldsymbol{x}_n), t_{\boldsymbol{w}_2}(\boldsymbol{x}_n), \ldots, t_{\boldsymbol{w}_L}(\boldsymbol{x}_n)]$$

$t_{\boldsymbol{w}}$ is some function with params $\boldsymbol{w}$, e.g., $t_{\boldsymbol{w}}(\boldsymbol{x}_n) = \cos(\boldsymbol{w}^\top \boldsymbol{x}_n)$

Core idea: For computational reasons, instead of the exact kernel with ϕ features map, we can use an approximation ψ

Approximating the expectation using an L sample based average with $\{\boldsymbol{w}_i\}_{i=1}^L$ drawn i.i.d. $p(\boldsymbol{w})$

$\boldsymbol{w}$ is a random draw from an appropriate distribution $p(\boldsymbol{w})$ like a zero mean Gaussian

Above summation is a dot product of two L-dim vectors

L (usually chosen to be small) is a hyperparam

*Random Features for Large-Scale Kernel Machines (Ben and Retch, NIPS 2007. Note: This paper actually won the test-of-time award at NIPS 2017)

Kernels (Will) Show Up in Deep Neural Nets Too!

For a deep neural network with params $\theta \in \mathbb{R}^P$, the output: $f_\theta(x)$

Can think of the gradient of $f_\theta(x)$ w.r.t. θ as "features" of input x

If $f_\theta(x) \in \mathbb{R}$ then $\phi_\theta(x) \in \mathbb{R}^P$

$$\phi_\theta(x) = \nabla_\theta f_\theta(x)$$

"Tangent" because ∇_θ defines a tangent

Dot product of such features defines the "Neural Tangent Kernel" (NTK)

$$k_{\text{NTK}}(x, z) = \nabla_\theta f_\theta(x)^\top \nabla_\theta f_\theta(z)$$

Intuition: Two inputs x and z are similar if parameter (θ) changes tend to change the network's outputs on them in similar ways.

We will discuss neural network later. For now, just remember that

Kernels = dot product between suitable feature representations of pairs of inputs

Kernels are not specific to just SVM or other "kernelizable" models but are relevant in "modern" deep learning too

